CHAPTER 1

INTRODUCTION

StudentSphere is a web-based educational platform developed using ReactJS, JavaScript, HTML, CSS, and MongoDB. It is designed to support college students by centralizing academic resources and offering continuous support through an interactive and user-friendly interface. This minor project aims to improve the learning process by providing access to subject-specific video content, downloadable study materials, and real-time doubt-solving via an AI-powered chatbot.

The core of StudentSphere is its Videos Page, where students can access a collection of educational videos related to their lab and semester subjects. These videos are contributed by both faculty and students, encouraging collaborative learning. In addition to original content, the platform integrates selected YouTube videos to broaden the learning scope with reliable external resources.

To facilitate access to written materials, the platform includes a Notes Section featuring Google Drive links to important documents such as PDFs. These resources can be downloaded easily, giving students the flexibility to study offline at their convenience.

Another standout feature of StudentSphere is the 24/7 AI Chatbot, which provides instant answers to students' common academic and campus-related queries. The chatbot enhances student support by offering real-time assistance, making it easier for students to stay informed and resolve doubts efficiently.

Built with a modern MERN stack, the platform ensures responsiveness, scalability, and efficient data management. Administrators can manage video content and notes to maintain the quality and relevance of materials available on the site. Students can register, log in, and personalize their learning experience through a streamlined interface.

With its blend of multimedia learning, downloadable content, and intelligent support, StudentSphere serves as a comprehensive educational companion for today's digitally connected students.

CHAPTER 2

LITERATURE SURVEY

The increasing reliance on digital education tools has led to the development of platforms that address various academic needs such as learning, resource sharing, and student support. The COVID-19 pandemic significantly accelerated this trend, necessitating online solutions for both learning and student engagement. The **StudentSphere** platform is designed in response to these evolving educational demands, offering a centralized solution for video-based learning, digital note sharing, and real-time student support via a chatbot.
This literature survey aims to present the development, benefits, challenges, and future trends in systems similar to StudentSphere, focusing on video-based learning platforms, digital resource distribution, and AI-driven educational support tools.

Development of Educational Resource Platforms

1. Evolution and Technologies

- **Traditional Learning Systems**: Early systems were confined to Learning Management Systems (LMS) such as Moodle and Blackboard, which provided limited interactivity and mostly text-based content.
- **Modern Learning Platforms**: Current systems leverage **web technologies (ReactJS, JavaScript)** and backend solutions like **MongoDB** to provide dynamic, scalable, and interactive platforms. Video content, user-contributed resources, and AI-powered features have become central.
- **Use of Cloud Storage**: To avoid the high costs of video hosting, platforms have adopted **cloud services like Google Drive** for storing and sharing multimedia and documents. This is especially beneficial for institutions and student-led initiatives with limited budgets.

2. Architectures and Frameworks

- **Web-Based Systems**: Platforms like StudentSphere are fully web-based, allowing access from any device with a browser. Built using **ReactJS**, the frontend ensures a responsive and intuitive user experience, while **MongoDB** supports flexible data storage for users, resources, and interactions.
- **Third-Party Integration**: Integration with tools like **YouTube and Google Drive** allows platforms to avoid hosting infrastructure while still providing rich educational content.

Advantages of StudentSphere-Like Systems

1. Content Accessibility and Collaboration

- **Peer and Faculty-Generated Content**: Students and teachers can both upload videos, enabling peer-assisted learning (PAL). This collaborative approach mirrors educational practices seen in platforms like YouTube EDU.
- **Cloud-Based Resources**: Notes and learning materials stored in Google Drive are easily downloadable, ensuring offline access and reducing dependence on platform storage.

2. Efficiency and Scalability

- **Low-Cost Deployment**: By avoiding commercial video hosting services, StudentSphere remains financially feasible for educational institutions.
- **Reusable Content**: Uploaded videos and notes can be accessed by multiple students across batches and years, maximizing the utility of each resource.

3. AI-Driven Student Support

- **24/7 AI Chatbot**: StudentSphere integrates an always-available chatbot that answers basic questions related to college schedules, departments, events, and student queries. This feature mirrors virtual assistants used in larger institutions and enhances the overall student experience.

Challenges and Limitations

1. Technical Constraints

- **Internet Dependency**: While videos and notes are accessible online, students in remote areas may struggle with connectivity issues that hinder learning.
- **Storage Limitations on Google Drive & Cloudinary**: While Drive & Cloudinary is a cost-saving option, it comes with storage quotas and sharing limits, which can become constraints as content volume increases.

2. Content Moderation and Quality

- **Verification of Uploaded Videos**: Ensuring the accuracy and academic value of peer-uploaded content requires administrative oversight to maintain quality and relevance.
- **Data Consistency**: Without a structured hosting backend, ensuring consistent metadata and categorization of content can be challenging.

3. Security and Privacy

- **Access Control**: Sharing notes and videos via external platforms like Google Drive may lead to unauthorized access if proper link management is not enforced.
- **User Data Protection**: Storing user details and interactions securely in MongoDB requires robust security practices to avoid data breaches.

Future Trends

1. AI and Adaptive Learning

- **Smarter Chatbots**: Future versions of the chatbot can incorporate natural language processing (NLP) and context awareness to provide more precise, adaptive responses.
- **Personalized Content Delivery**: Machine learning algorithms can be used to recommend videos and notes based on student behaviour and performance.

2. Community-Driven Education

- **Decentralized Learning**: Platforms like StudentSphere promote a shift from instructor-led models to community-driven learning, where students are both consumers and creators of content.

3. Analytics and Feedback

- **Learning Analytics**: Integrating feedback loops and basic analytics can help students track their progress and engagement levels with different types of content.

Conclusion

StudentSphere presents an innovative approach to addressing the digital learning needs of college students by combining peer-driven video content, easily accessible notes, and AI-enabled support—all on a cost-efficient platform built with modern web technologies. While there are challenges around scalability, moderation, and access, the model offers a scalable, community-oriented learning ecosystem. Ongoing technological advancements in AI, cloud integration, and web development will continue to enhance platforms like StudentSphere, making education more accessible, personalized, and interactive.

CHAPTER 3
SOFTWARE REQUIREMENTS ANALYSIS

Purpose

StudentSphere is a collaborative web platform designed to enhance academic resource sharing among students and faculty. It facilitates the upload and access of educational content, including videos and notes, while providing AI-driven assistance for academic queries. The platform acts as an enabler—not a content creator—with verified users contributing and accessing materials within a secure, moderated environment.

Scope

This document defines the functional and non-functional requirements for StudentSphere. It guides the development and testing teams throughout the software lifecycle. The system replaces informal communication channels with a structured, role-based platform that ensures accessibility, collaboration, and academic integrity for all registered users.

3.1 ANALYSIS MODEL

Development Approach: Agile Methodology

The StudentSphere project adheres to the Agile development methodology, emphasizing iterative delivery, continuous feedback, and adaptability to change. Agile empowers cross-functional collaboration among developers, designers, and stakeholders, ensuring that features are incrementally built and refined based on actual user needs.
Each Agile sprint involves the following phases:

- Requirement Elicitation and Prioritization
- Sprint Planning and Story Estimation
- Incremental Design and Development
- Unit and Integration Testing
- Sprint Review and Retrospective

This model facilitates early identification of issues, promotes high-quality delivery, and ensures that the software evolves with user expectations throughout its lifecycle.

3.2 EXISTING SYSTEM

In conventional academic settings, resource sharing, student engagement, and academic support are often disjointed and inefficient. Institutions typically rely on fragmented systems (e.g., physical notes, informal chat groups, or separate storage services), which results in poor discoverability, limited interactivity, and high administrative overhead.

3.2.1 Challenges in the Existing System

- **Decentralized Content Distribution**
  Educational materials are scattered across multiple platforms, making it difficult for students to access them in a unified and structured manner.
- **Manual Query Resolution**
  Students rely on in-person or manual digital communication for clarifications, which causes delays and inconsistent support.
- **Limited Student Involvement**
  Students rarely contribute content, and when they do, it lacks structure and moderation.
- **Lack of Intelligent Assistance**
  No automated systems are in place to provide real-time academic or institutional support.

3.3 PROPOSED SYSTEM

StudentSphere is designed to be a collaborative, intelligent, and centralized academic portal that addresses the limitations of current systems. Built with a modern technology stack—**ReactJS**, **JavaScript**, **HTML/CSS**, and **MongoDB**—the system supports content uploading, AI-powered interactions, and secure content management, all within a user-friendly web interface.

Core Objectives

- Centralize academic resources (videos, notes, etc.) for easy access.
- Empower both faculty and students to contribute and share content.
- Implement an intelligent chatbot for real-time query handling.
- Ensure content integrity through administrative moderation.

System Constraints

- Video and document hosting will rely on **Google Drive** to reduce server and storage costs.
- The system requires continuous internet access for full functionality, especially for chatbot and cloud-hosted content.

Assumptions

- Users possess basic digital literacy and familiarity with academic platforms.
- Uploaded content adheres to institutional guidelines and is verified by the administrator.
- User devices support modern browsers for optimal UI rendering.

Dependencies

- Dependency on **third-party APIs** (e.g., OpenAI for chatbot functionality).
- Google Drive APIs for content management and delivery.
- A stable internet connection for access to cloud-based services.

3.4 STUDY OF THE SYSTEM

3.4.1 Number of Modules

1. The system is logically divided into the following modules:
2. **Admin Module**
   - Content moderation and approval workflow
   - User management (faculty and students)
   - Analytics and system configuration
3. **Faculty Module**
   - Upload and manage academic videos and notes
   - Track student engagement metrics
   - Share Google Drive links for document access
4. **Student Module**
   - View and search academic materials
   - Upload content for review
   - Engage with chatbot for academic and administrative assistance
5. **Chatbot Module**
   - AI-based natural language interaction
   - Handles FAQs on syllabus, schedules, exam patterns, etc.
   - Escalation to admin in case of unresolved queries

3.5 FUNCTIONAL REQUIREMENTS

- **User Authentication and Authorization**
  - Secure registration and login with role-based access (admin, faculty, student)
  - Password encryption and validation
- **Content Upload and Moderation**
  - Video and document submission interface for faculty and students

    - Admin approval pipeline to maintain academic quality
    - Metadata tagging for improved search and categorization
- **Interactive Dashboard**
    - Personalized views for each user role
    - Summary of uploads, pending approvals, and chatbot activity
- **AI-Powered Chatbot**
    - Real-time assistance on academic queries
    - Natural Language Processing for conversational understanding
    - Integration with backend knowledge base
- **Search and Discovery**
    - Filterable search based on subject, semester, or keyword
    - View counts and recommendations for trending content

3.6 NON-FUNCTIONAL REQUIREMENTS

- **Usability**
    - Responsive design using **Tailwind CSS** and **Framer Motion** for modern UI/UX
    - Accessibility support for differently-abled users
- **Security**
    - Encrypted data transmission using HTTPS
    - Strict access controls to prevent unauthorized content modification
- **Performance**
    - Optimized loading of Google Drive content using lazy loading techniques
    - Backend API caching to reduce response times
- **Maintainability**
    - Modular and clean codebase to facilitate future enhancements
    - Comprehensive documentation for developers and maintainers
- **Scalability**
    - Designed for horizontal scaling to support increasing users and content
    - Stateless backend architecture for easy deployment
- **Availability and Reliability**
    - 24/7 access to the system with minimal planned downtime
    - Daily data backups and recovery mechanisms in place
- **Portability**
    - Cross-platform compatibility across Windows, Linux, and macOS
    - Works seamlessly on modern browsers and devices

CHAPTER 4
SOFTWARE DESIGN

Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm and area of application. Design is the first step in the development phase for any engineered product or system. The designer's goal is to produce a model or representation of an entity that will later be built. Beginning, once system requirement have been specified and analyzed, system design is the first of the three technical activities design, code and test that is required to build and verify software.

During design, progressive refinement of the data structure, program structure, and procedural details are developed reviewed and documented. System design can be viewed from either technical or project management perspective. From the technical point of view, design is comprised of four activities - architectural design, data structure design, **interface** design and procedural design.

E-R DIAGRAMS

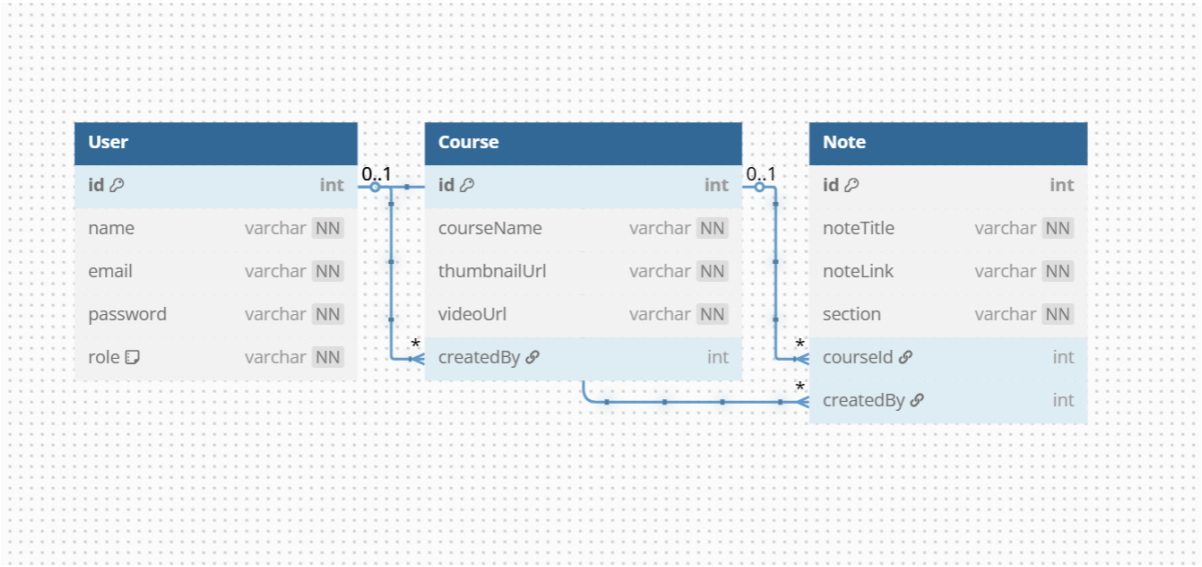
The relation upon the system is structure through a conceptual ER-Diagram, which not only specifies the existential entities but also the standard relations through which the system exists and the cardinalities that are necessary for **the** system state to continue.

The entity Relationship Diagram (ERD) depicts the relationship between the data objects. The ERD is the notation that is used to conduct the date modelling activity the attributes of each data object noted is the ERD can be described resign a data object descriptions.

The set of primary components that are identified by the ERD are:

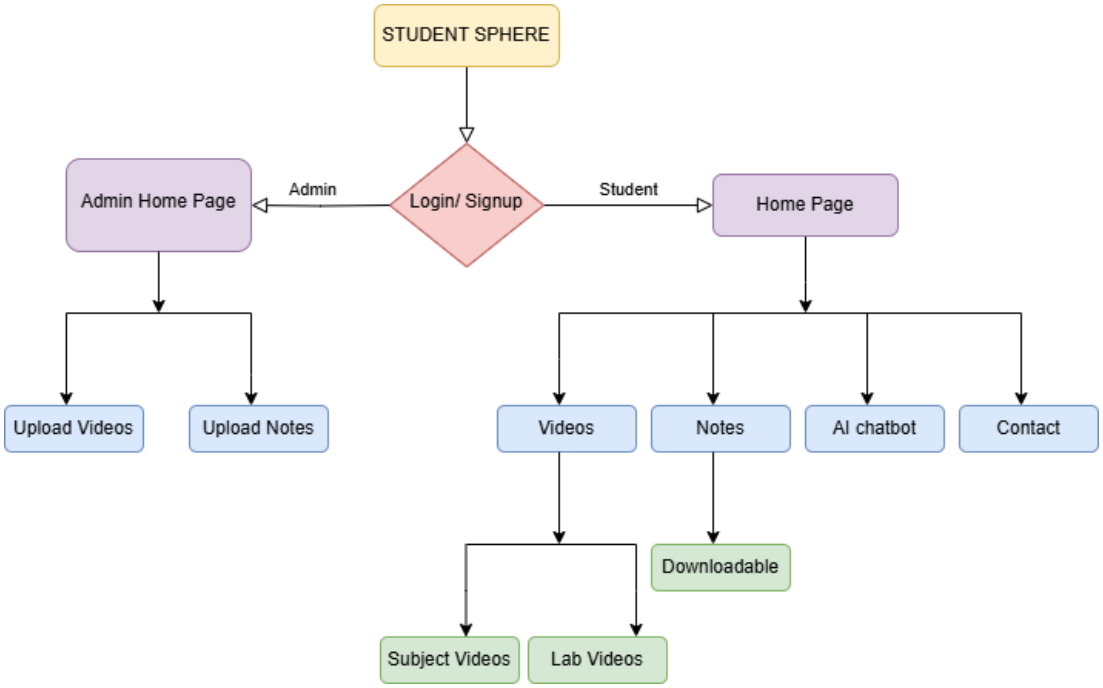
- Data object
- Relationship
- Attributes
- Various types of indicators

4.1 ER- Diagram:



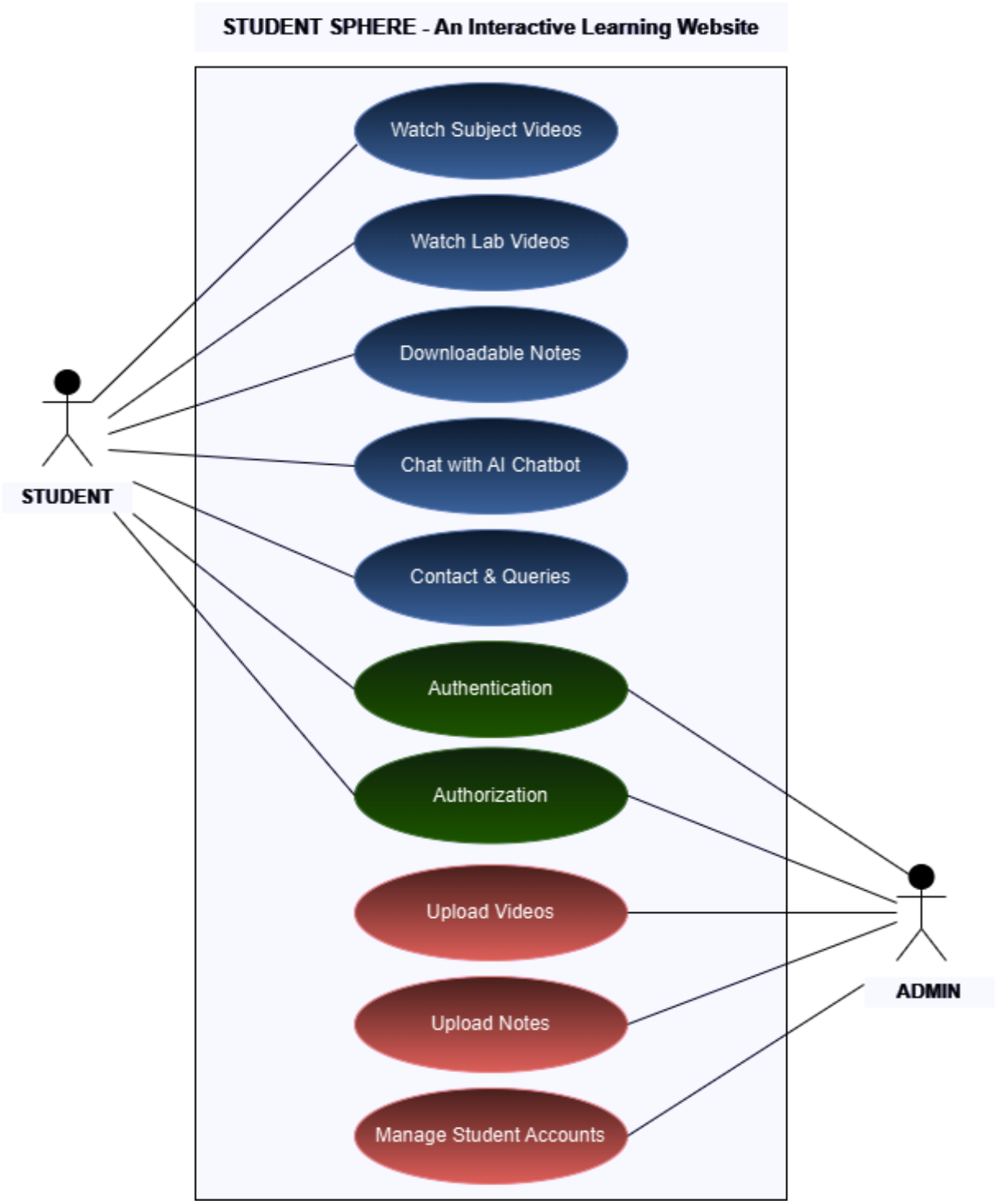
Figno: 4.1

4.2 Flow Chart:



Figno: 4.2

4.3 Use Case Diagram:



Figno: 4.3

CHAPTER 5

SOFTWARE AND HARDWARE REQUIREMENTS

5.1 HARDWARE REQUIREMENTS

The hardware configuration plays a crucial role in ensuring smooth and efficient system performance. Adequate processing power and memory are essential to handle concurrent users, content access, and data operations.

Client Side (User Devices):

Processor	**:** Dual-core 2.0 GHz or higher
RAM	**:** 4 GB minimum
Storage	**:** 500 MB free space for browser cache
Display	**:** Minimum 1024x768 resolution
Internet	**:** Stable broadband connection for video streaming

Server Side (Deployment Environment):

Processor	: Quad-core 2.5 GHz or higher
RAM	: 8 GB or more
Storage	: Minimum 100 GB (expandable based on data volume)
Network	: High-speed internet with SSL support
Cloud Support	: Integration with Google Drive for content hosting

5.2 SOFTWARE REQUIREMENTS

The StudentSphere application is built using a modern full-stack JavaScript framework, ensuring responsiveness, scalability, and cross-platform compatibility. The system is optimized for both academic scalability and user-friendly interaction.

Frontend Technologies:

Framework	**:** ReactJS
Languages	**:** JavaScript
Markup & Styling	**:** HTML5, CSS3, Tailwind CSS
Styling	**:** Tailwind CSS
Animation	**:** Framer Motion

Backend Technologies:

Database	**:** MongoDB (NoSQL, cloud-hosted via MongoDB Atlas)
Server Framework	**:** Node.js with Express.js
Authentication	**:** JWT-based secure login system

Other Tools and Services:

Content Storage	**:** Google Drive & Cloudinary API integration for storing and streaming
Version Control	**:** Git with GitHub
Development Platform	**:** Visual Studio Code
Operating System	**:** Cross-platform (Windows/macOS/Linux supported)
Package Manager	**:** npm or yarn
Deployment	**:** Vercel or Netlify (Frontend), Render/Heroku (Backend)

CHAPTER 6
CODING

6.1 SOFTWARE ENVIRONMENT

JavaScript

JavaScript is a high-level, interpreted programming language that is widely used for web development to create dynamic and interactive user interfaces. Originally developed by Netscape, it has become the standard scripting language supported by all modern web browsers.

Key Features and Concepts:

1. Client-Side Scripting: JavaScript runs primarily in web browsers, enabling interactive web pages without the need for server communication.
2. Event-Driven Programming: It allows developers to execute code in response to user actions such as clicks, typing, or page loads.
3. Dynamic Typing: JavaScript is loosely typed, meaning variables can hold different data types during execution.
4. Prototypal Inheritance: JavaScript uses prototypes instead of classical inheritance, allowing flexible object creation and reuse.
5. Asynchronous Programming: Supports asynchronous operations through callbacks, promises, and async/await, enabling non-blocking code execution.
6. Wide Ecosystem: A vast collection of libraries and frameworks built on JavaScript, like React.js, enhances development efficiency.
7. Cross-Platform: Runs on any device with a compatible web browser or Node.js runtime for server-side use.

USAGE:

JavaScript is used extensively in web development to build responsive and interactive websites. It is also used on the server side with Node.js, mobile app development through frameworks like React Native, and even in desktop applications.

React.js

React.js is a popular open-source JavaScript library developed by Facebook for building user interfaces, particularly single-page applications where a seamless user experience is critical.

Key Features and Concepts:

1. Component-Based Architecture: React breaks down the UI into reusable components, making the code modular and easier to maintain.
2. Virtual DOM: React uses a virtual representation of the DOM to optimize and minimize real DOM manipulations, improving performance.
3. Declarative UI: Developers describe what the UI should look like, and React manages the rendering efficiently.
4. One-Way Data Binding: Data flows in a single direction, making the application more predictable and easier to debug.
5. JSX Syntax: React uses JSX, a syntax extension that allows HTML-like code inside JavaScript, simplifying UI definition.
6. Strong Community & Ecosystem: Rich ecosystem including libraries for routing (React Router), state management (Redux), and more.

USAGE:
React.js is ideal for building dynamic, high-performance web applications such as dashboards, social networks, and educational platforms like StudentSphere.

MongoDB

MongoDB is a NoSQL, document-oriented database designed to store data in flexible, JSON-like documents, allowing for easy scalability and rapid development.

Key Features and Concepts:

1. Schema-less Database: Unlike traditional relational databases, MongoDB allows dynamic schemas, enabling developers to store varying data structures.
2. High Performance: Supports fast read/write operations, indexing, and replication for availability.
3. Scalability: Designed for horizontal scaling via sharding, handling large data volumes efficiently.
4. Rich Query Language: Supports powerful queries, aggregation pipelines, and full-text search.
5. Flexible Data Model: Ideal for applications with evolving data requirements or unstructured data.
6. Built-in Replication & High Availability: Supports replica sets for automatic failover and data redundancy.

USAGE:
MongoDB is widely used for web applications requiring flexible data models and scalability. It fits perfectly with JavaScript-based stacks like MERN (MongoDB, Express, React, Node.js) used in StudentSphere.

Express.js

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features to develop web and mobile applications.

Key Features and Concepts:

1. Middleware Support: Express uses middleware functions to handle requests, responses, and routing logic efficiently.
2. Routing: Provides a simple yet powerful routing system for handling HTTP requests.
3. Extensibility: Easily integrates with databases, templating engines, and other tools.
4. Minimalist: Focuses on simplicity and performance without unnecessary overhead.
5. RESTful APIs: Frequently used to build RESTful web services and APIs.

USAGE:

Express.js forms the backend framework for StudentSphere, managing server-side logic, API endpoints, and database interactions.

Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a browser, primarily on servers.

Key Features and Concepts:

1. Event-Driven, Non-Blocking I/O: Node.js uses asynchronous programming and event-driven architecture to handle multiple connections efficiently.
2. Single-Threaded but Scalable: Though single-threaded, it can handle concurrency via event loops and worker threads.
3. V8 Engine: Powered by Google's V8 JavaScript engine for fast code execution.
4. Rich Package Ecosystem: Access to npm (Node Package Manager) provides thousands of reusable packages and modules.
5. Full-Stack JavaScript: Enables use of JavaScript on both client and server sides, simplifying development.

USAGE:

Node.js is used in StudentSphere for building scalable server-side applications, handling API requests, and managing database communication.

6.1 SOURCE CODE

Admin Courses:

```
import { toast } from "sonner";
import { useEffect, useRef, useState } from "react";
import { Loader2, Pencil, Plus, Trash, X } from "lucide-react";
import { getCourses, deleteCourses, editCourses } from "../../api/api";

const AdminCourses = () => {
  const [courses, setCourses] = useState(null);
  const [loading, setLoading] = useState(true);
  const [adding, setAdding] = useState(false);
  const [currentProduct, setCurrentProduct] = useState(null);
  const [showAdd, setShowAdd] = useState(false);
  const [showEdit, setShowEdit] = useState(false);
  const [selectedVideo, setSelectedVideo] = useState(null);

  const courseNameRef = useRef("");
  const [thumbnailFile, setThumbnailFile] = useState(null);
  const [videoFile, setVideoFile] = useState(null);

  const uploadToCloudinary = async (file, resourceType) => {
    const formData = new FormData();
    formData.append("file", file);
    formData.append(
      "upload_preset",
      import.meta.env.VITE_CLOUDINARY_UPLOAD_PRESET
    );

    const res = await fetch(
      `https://api.cloudinary.com/v1_1/${
        import.meta.env.VITE_CLOUDINARY_CLOUD_NAME
      }/${resourceType}/upload`,
      {
        method: "POST",
        body: formData,
      }
    );

    const data = await res.json();

    if (!res.ok) throw new Error(data.error?.message || "Upload failed");
```

```
  return data.secure_url;
};

const fetchData = async () => {
  try {
    const res = await getCourses();
    if (res.status === 200) {
      setCourses(res.data);
    }
  } catch (error) {
    console.error(error);
  } finally {
    setLoading(false);
  }
};

const handleAdd = async (e) => {
  e.preventDefault();
  setAdding(true);

  try {
    const thumbnailUrl = await uploadToCloudinary(thumbnailFile, "image");

    const formData = new FormData();
    formData.append("courseName", courseNameRef.current.value);
    formData.append("thumbnailUrl", thumbnailUrl);
    formData.append("video", videoFile);

    const res = await fetch("http://localhost:3000/courses/add", {
      method: "POST",
      body: formData,
    });

    if (!res.ok) {
      const errorData = await res.json();
      throw new Error(errorData.error || "Failed to add course");
    }

    const data = await res.json();
    setShowAdd(false);
    toast.success("Course Added");
    setCourses((prevCourses) => [...prevCourses, data.course]);
```

```
  } catch (error) {
    toast.error("Error while Adding");
    console.error("error while adding", error);
  } finally {
    setAdding(false);
  }
};

const editHelper = (product) => {
  setCurrentProduct(product);
  setShowEdit(true);
};

const handleEdit = async (e) => {
  e.preventDefault();

  const product = {
    courseName: courseNameRef.current.value,
    thumbnailUrl: currentProduct.thumbnailUrl,
    videoUrl: currentProduct.videoUrl,
  };

  try {
    const response = await editCourses(product, currentProduct._id);
    if (response.status === 200) {
      setShowEdit(false);
      fetchData();
      toast.success("Course Updated");
    }
  } catch (error) {
    toast.error("Error while Updating");
    console.error("Error while editing", error);
  }
};

const handleDelete = async (id) => {
  try {
    const response = await deleteCourses(id);
    if (response.status === 200) {
      fetchData();
      toast.success("Course Deleted");
    }
  } catch (error) {
```

```
    toast.error("Error while Deleting");
    console.error("Error while deleting", error);
  }
 };

 const convertToPlayableURL = (url) => {
  const match = url.match(/drive\.google\.com\/file\/d\/(.+?)\//);
  return match ? `https://drive.google.com/file/d/${match[1]}/preview` : url;
 };

 useEffect(() => {
  fetchData();
 }, []);

 if (loading) {
  return (
   <div className="w-screen h-[90vh] flex flex-col justify-center items-center">
    <Loader2 className="text-lime-500 h-14 w-14 animate-spin" />
   </div>
  );
 }

 return (
  <div className="w-full flex flex-col justify-start items-start px-2 md:px-6">
   <div className="w-full flex flex-row justify-between items-center my-4 shadow-md
rounded-md p-1 border">
    <button
     className="w-10 h-10 font-bold flex justify-center items-center border-2 border-
green-500 rounded-md text-green-500 shadow-md hover:text-white hover:bg-green-500
hover:shadow-md hover:shadow-green-400"
     onClick={() => setShowAdd(!showAdd)}
    >
     <Plus className="w-8 h-8" />
    </button>
   </div>

   {/* Responsive wrapper for table with horizontal scroll on small screens */}
   <div className="w-full overflow-x-auto">
    <table className="min-w-[600px] md:min-w-full border-collapse border shadow-lg
rounded-md">
     <thead className="shadow-md font-bold text-lime-500 text-left rounded-md bg-
lime-100">
      <tr>
```

```
    <th className="p-4 md:p-6">PID</th>
    <th className="p-4 md:p-6">Course Name</th>
    <th className="p-4 md:p-6">Thumbnail</th>
    <th className="p-4 md:p-6">Video</th>
    <th className="p-4 md:p-6">Actions</th>
  </tr>
</thead>
<tbody>
  {courses?.map((product, index) => (
    <tr
      key={index}
      className="border-b last:border-none hover:bg-lime-50 transition-colors"
    >
      <td className="p-2 md:p-4 break-all">{product._id}</td>
      <td className="p-2 md:p-4 break-words max-w-[150px] md:max-w-none">
        {product.courseName}
      </td>
      <td className="flex justify-start px-2 md:px-4 items-center">
        <img
          src={product.thumbnailUrl}
          alt={product.title}
          className="h-12 w-12 object-cover rounded-full shadow-md bg-lime-500"
        />
      </td>
      <td
        className="p-2 md:p-4 text-blue-600 underline cursor-pointer break-all max-w-
            [150px] md:max-w-none"
        onClick={() => setSelectedVideo(product.videoUrl)}
      >
        {product.videoUrl}
      </td>
      <td className="p-2 md:p-4 flex flex-wrap gap-2 md:gap-4">
        <button
          className="border-blue-500 border-2 p-1 rounded-md text-blue-500 shadow-
              md hover:bg-blue-500 hover:text-white flex items-center justify-center"
          onClick={() => editHelper(product)}
          aria-label={`Edit course ${product.courseName}`}
        >
          <Pencil />
        </button>
        <button
          className="border-red-500 border-2 p-1 rounded-md text-red-500 shadow-md
              hover:bg-red-500 hover:text-white flex items-center justify-center"
```

```
            onClick={() => handleDelete(product._id)}
            aria-label={`Delete course ${product.courseName}`}
           >
            <Trash />
           </button>
          </td>
         </tr>
        ))}
       </tbody>
      </table>
     </div>

     {/* Add Modal */}
     {showAdd && (
      <div className="fixed top-0 left-0 z-50 h-screen w-screen flex justify-center items-
          center bg-black/40 p-4">
       <div className="w-full max-w-md bg-white shadow-2xl rounded-md flex flex-col
           items-center max-h-[90vh] overflow-y-auto">
        <div className="w-[90%] flex justify-between items-center mt-4">
         <h1 className="text-xl font-bold text-green-500">Add Course</h1>
         <X
          className="text-red-500 cursor-pointer"
          onClick={() => setShowAdd(false)}
         />
        </div>
        <form
         className="w-[90%] flex flex-col gap-4 mt-6"
         onSubmit={handleAdd}
        >
         <input
          ref={courseNameRef}
          type="text"
          placeholder="Course Name"
          required
          className="p-2 bg-[#f5f5f7] border-b-2 focus:border-lime-400 outline-none
          rounded-sm w-full"
         />
         <input
          type="file"
          accept="image/*"
          onChange={(e) => setThumbnailFile(e.target.files[0])}
          required
          className="w-full"
```

```
            />
            <input
              type="file"
              accept="video/*"
              onChange={(e) => setVideoFile(e.target.files[0])}
              required
              className="w-full"
            />
            <button
              type="submit"
              disabled={adding}
              className={`h-12 rounded-md shadow-md text-white flex items-center justify-
                center gap-2 w-full
                ${
                  adding
                    ? "bg-gray-400 cursor-not-allowed"
                    : "bg-green-500 hover:bg-green-600"
                }`}
            >
              {adding && <Loader2 className="animate-spin h-5 w-5" />}
              {adding ? "Uploading..." : "Add"}
            </button>
          </form>
        </div>
      </div>
    )}

    {/* Video Modal */}
    {selectedVideo && (
      <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 p-4">
        <div className="bg-white p-4 rounded-lg shadow-lg w-full max-w-4xl relative max-
h-[90vh] overflow-y-auto">
          <button
            className="absolute top-2 right-2 text-red-500 hover:bg-red-500 hover:text-white
border border-red-500 rounded-full p-1"
            onClick={() => setSelectedVideo(null)}
            aria-label="Close video"
          >
            <X />
          </button>
          {selectedVideo.includes("drive.google.com") ? (
            <iframe
              src={convertToPlayableURL(selectedVideo)}
```

```
              width="100%"
              height="500px"
              allow="autoplay"
              frameBorder="0"
              title="Google Drive Video"
              allowFullScreen
             />
            ) : (
             <video
              controls
              width="100%"
              height="500px"
              className="rounded-md max-w-full"
              src={selectedVideo}
             />
            )}
           </div>
          </div>
        )}

        {/* Edit Modal */}
        {showEdit && (
         <div className="fixed top-0 left-0 z-50 h-screen w-screen flex justify-center items-
center bg-black/40 p-4">
          <div className="w-full max-w-md bg-white shadow-2xl rounded-md flex flex-col
items-center max-h-[80vh] overflow-y-auto">
           <div className="w-[90%] flex justify-between items-center mt-4">
            <h1 className="text-xl font-bold text-green-500">Edit Course</h1>
            <X
             className="text-red-500 cursor-pointer"
             onClick={() => setShowEdit(false)}
            />
           </div>
           <form
            className="w-[90%] flex flex-col gap-4 mt-6"
            onSubmit={handleEdit}
           >
            <input
             ref={courseNameRef}
             defaultValue={currentProduct?.courseName}
             type="text"
             placeholder="Course Name"
             required
```

```
            className="p-2 bg-[#f5f5f7] border-b-2 focus:border-lime-400 outline-none
rounded-sm w-full"
          />
          {/* Optionally add inputs to update thumbnail/video here */}
          <button
            type="submit"
            className="h-12 rounded-md bg-green-500 shadow-md text-white hover:bg-
green-600 w-full"
          >
            Save
          </button>
        </form>
      </div>
    </div>
  )}
 </div>
 );
};

export default AdminCourses;
```

- **CoursesRoute.js**

```
const express = require("express");
const multer = require("multer");
const { Readable } = require("stream");
const cloudinary = require("../utils/cloudinary");
const Course = require("../models/courses");
const router = express.Router();

const storage = multer.memoryStorage();
const upload = multer({ storage });

// Create a new course with video upload
router.post("/add", upload.single("video"), async (req, res) => {
  try {
    const { courseName, thumbnailUrl } = req.body;
    const file = req.file;

    if (!file) {
      return res.status(400).json({ error: "No video file uploaded" });
    }

    // Upload video to Cloudinary
    const uploadStream = cloudinary.uploader.upload_stream(
      { resource_type: "video", folder: "courses" },
      async (error, result) => {
        if (error) {
          console.error("Cloudinary upload error:", error);
          return res.status(500).json({ error: "Cloudinary upload failed" });
        }

        // Save course with video URL
        const course = new Course({
          courseName,
          thumbnailUrl,
          videoUrl: result.secure_url,
        });

        await course.save();
        res.status(201).json({ message: "Course added successfully!", course });
      }
    );
```

```
    // Create a readable stream from multer buffer and pipe to Cloudinary
    const bufferStream = new Readable();
    bufferStream.push(file.buffer);
    bufferStream.push(null);

    bufferStream.pipe(uploadStream);
  } catch (error) {
    console.error("Server error:", error);
    res.status(500).json({ error: "Failed to add course" });
  }
});

// Get all courses
router.get("/", async (req, res) => {
  try {
    const courses = await Course.find();
    res.status(200).json(courses);
  } catch (error) {
    res.status(500).json({ error: "Failed to fetch courses" });
  }
});

// Get a single course by ID
router.get("/:id", async (req, res) => {
  try {
    const course = await Course.findById(req.params.id);
    if (!course) {
      return res.status(404).json({ message: "Course not found" });
    }
    res.status(200).json(course);
  } catch (error) {
    res.status(500).json({ error: "Failed to fetch course" });
  }
});

// Update a course
router.put("/edit/:id", async (req, res) => {
  try {
    const { courseName, thumbnailUrl, videoUrl } = req.body;
    const updatedCourse = await Course.findByIdAndUpdate(
      req.params.id,
      { courseName, thumbnailUrl, videoUrl },
      { new: true }
```

```
    );
    if (!updatedCourse) {
      return res.status(404).json({ message: "Course not found" });
    }
    res.status(200).json({ message: "Course updated successfully!", updatedCourse });
  } catch (error) {
    res.status(500).json({ error: "Failed to update course" });
  }
});

// Delete a course
router.delete("/delete/:id", async (req, res) => {
  try {
    await Course.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: "Course deleted successfully" });
  } catch (error) {
    res.status(500).json({ error: "Failed to delete course" });
  }
});

module.exports = router;
```

CHAPTER 7

SYSTEM TESTNG AND IMPLEMENTATION

Software testing is a critical element of software quality assurance and represents the ultimate review of specifications, design, and coding. In fact, testing is the one step in the software engineering process that could be viewed as destructive rather than constructive.

A strategy for software testing integrates software test case design methods into a well-planned series of steps that result in the successful implementation of software. Testing is a set of activities that can be planned in advance and conducted systematically. The underlying motivation of program testing is to affirm software quality through methods that can economically and effectively apply to both large and small-scale systems.

7.1. STRATEGIC APPROACH TO SOFTWARE TESTING

The software engineering process can be visualized as a spiral. Initially, system engineering defines the role of software and leads to software requirement analysis, where the information domain, functionality, behavior, performance, constraints, and validation criteria for software are established. Moving inward along the spiral, we move from design to coding.

A strategy for software testing follows the same spiral structure. Testing begins at the core with unit testing, proceeds outward to integration testing, then to validation testing, and finally reaches system testing. In the case of StudentSphere, this approach ensures each module—such as user login, video streaming, chatbot integration, and document download—is tested independently and collectively as part of the full system.

7.2. TESTING

1. Unit Testing

Unit testing focuses on the smallest functional components of StudentSphere, such as:

- ReactJS components like VideoCard, ChatBot, and NotesSection.
- API endpoints for user authentication, content upload, and retrieval.
- Utility functions such as file validation or date formatting.

Tools Used: Jest, React Testing Library, Postman (for backend routes).

2. White Box Testing

This type of testing ensures:

- All independent code paths are tested (e.g., login success/failure).
- All logical decisions (e.g., video visibility based on user type) are tested for both true and false conditions.
- All loops (mapping over video lists) are tested at minimum, maximum iterations.

- Internal structures like form states and Redux store updates are verified.

Application in StudentSphere: Each component was individually tested to confirm correct data flow, conditions, and event handling.

3. Basic Path Testing

We employed flow graphs to map out logical sequences for functions like:

- Chatbot interaction flow.
- Admin uploading resources.
- Students navigating from dashboard to a specific subject's video.

Cyclomatic Complexity Calculation:

For example, the chatbot response logic was mapped and tested using:

$V(G) = E - N + 2$

$V(G) = P + 1$

Where E = edges, N = nodes, and P = predicate nodes.

4. Conditional Testing

Each major conditional branch—like whether a user is admin or student, or whether a video is approved—was tested for both outcomes. This ensures that each scenario results in the expected view or functionality.

Example Scenarios:

- If user is student → Display video only
- If user is admin → Display video with edit/delete options

5. Data Flow Testing

Variables like userSession, videoList, and chatHistory were tested across components:

- Confirmed where they were declared, initialized, and used.
- Checked for improper overwrites or uninitialized accesses.
- This was critical in nested functions within the chatbot component.

6. Loop Testing

Loop testing was applied to:

- Iteration over video listings (map() over arrays).
- Chatbot response loops based on keyword matching.
- Dynamic rendering of notes and drive links.

Loop Conditions Tested:

- Zero items (e.g., "No videos uploaded yet.")
- Single item
- Multiple items
- Skipping iterations (e.g., filtering hidden/unapproved videos)

7.3. SYSTEM TESTING AND IMPLEMENTATION

System testing for StudentSphere involved validating complete, end-to-end scenarios that reflect actual user behaviour. This included interaction between all modules—frontend components, backend services, MongoDB integration, and third-party APIs like YouTube and Google Drive.

Test Environment:

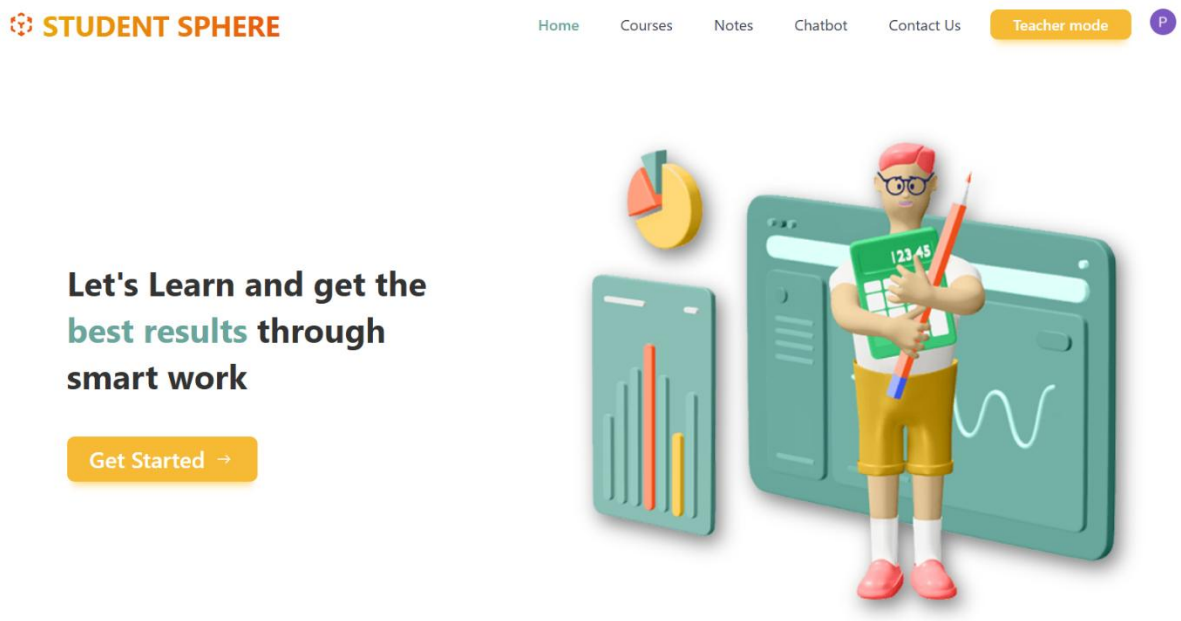
- Frontend hosted on localhost or Vercel.
- Backend server with MongoDB Atlas.

7.3.1 TEST CASES:

Test Case	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Pass/ Fail
1	Launch Application	Open browser and access homepage	URL: localhost:3000	Dashboard loads with login/register options	Loaded successfully	Pass
2	User Login (Valid)	Fill login form with valid credentials	user123 / pass123	Redirect to student dashboard	Redirected correctly	Pass
3	User Login (Invalid)	Enter wrong password	user123 / wrong pass	Show error message	Error shown	Pass
4	Video Access	Navigate to Videos section	-	Display videos with preview cards	Videos rendered	Pass
5	Notes Download	Open Notes tab and click download	Google Drive PDF	File starts downloading	Download successful	Pass
6	Chatbot Query	Ask: “How to upload videos?”	Text input	Chatbot replies with guidance	Correct response displayed	Pass
7	Admin Upload	Login as admin→ Upload new	File + metadata	Video added to system	Appears in list	Pass

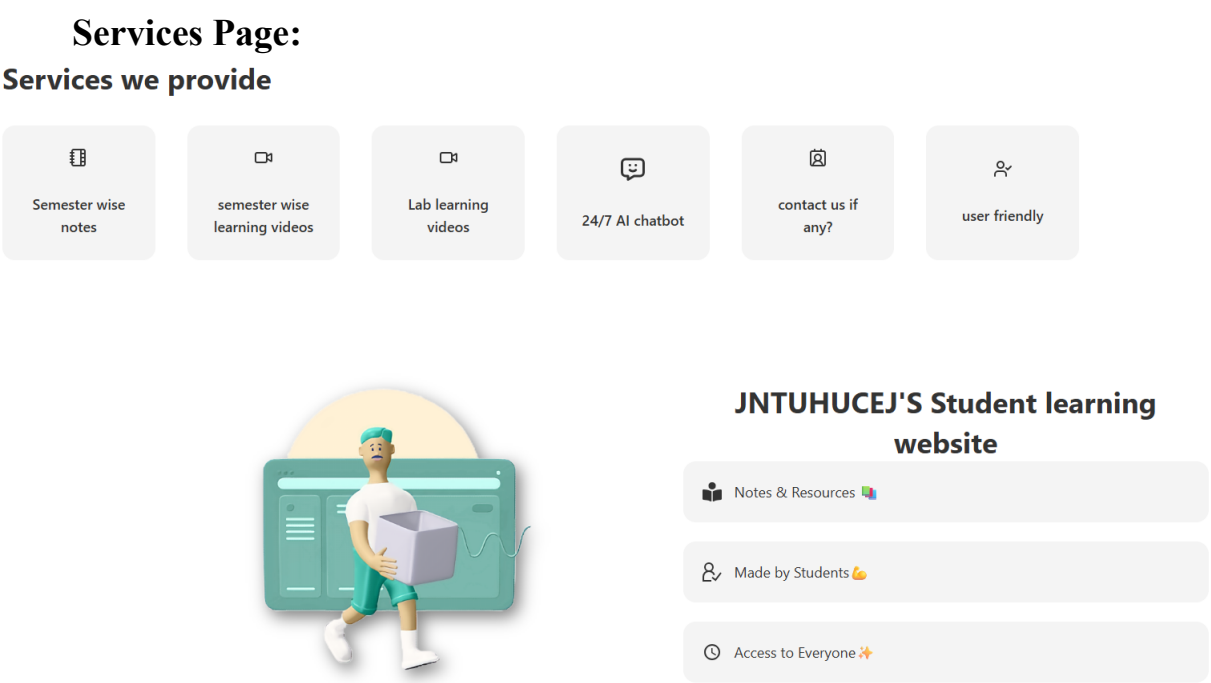
CHAPTER 8
OUTPUT SCREENS

Home Page:



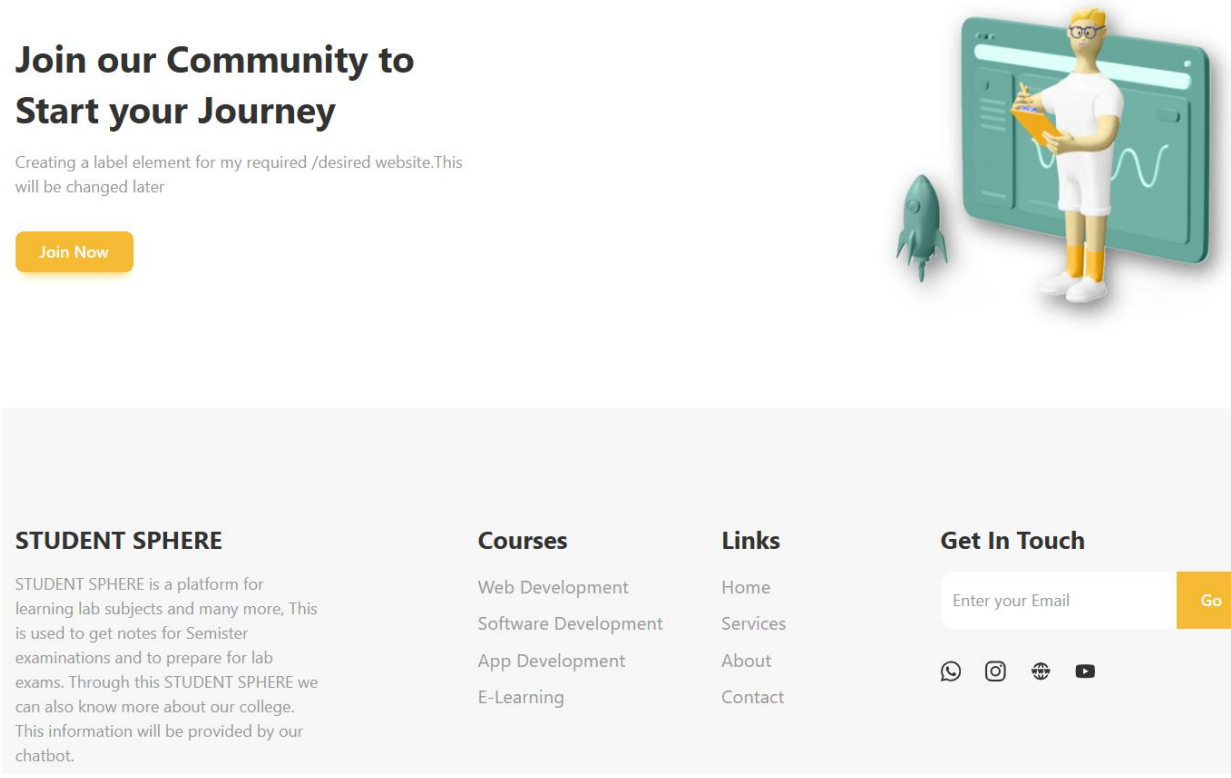
Figno:8.1

Services Page:



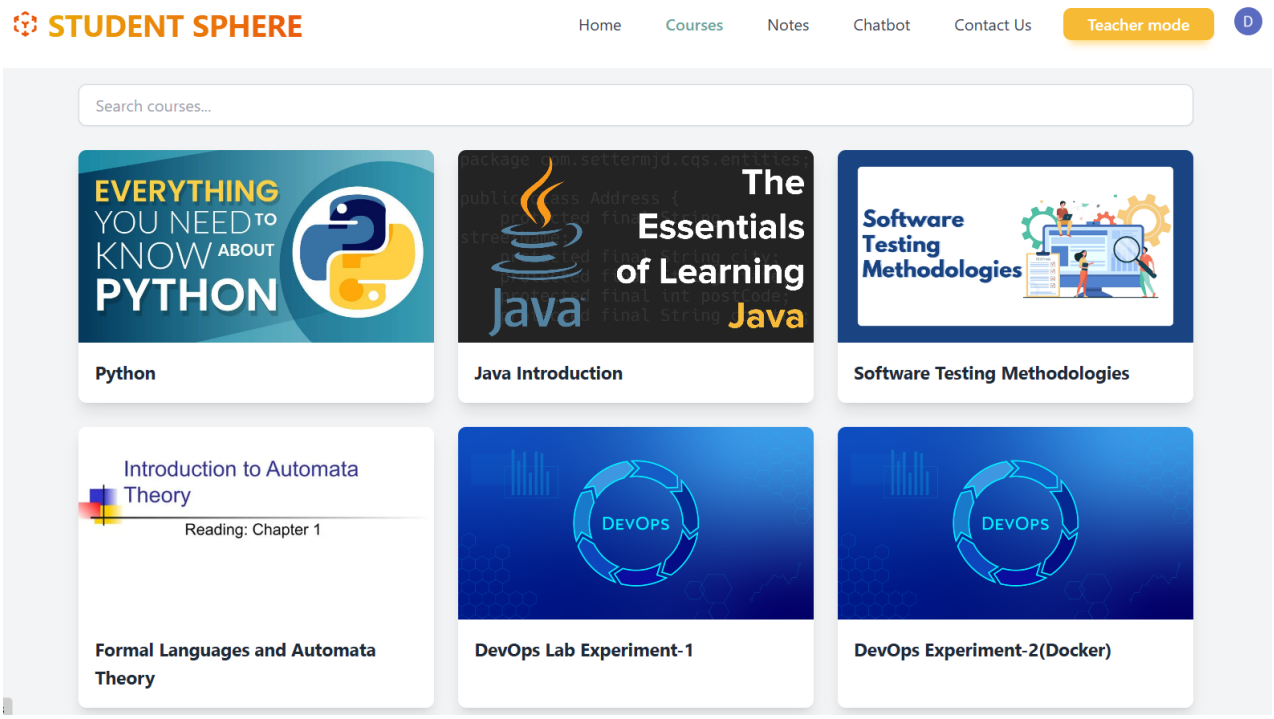
Figno:8.2

Footer:



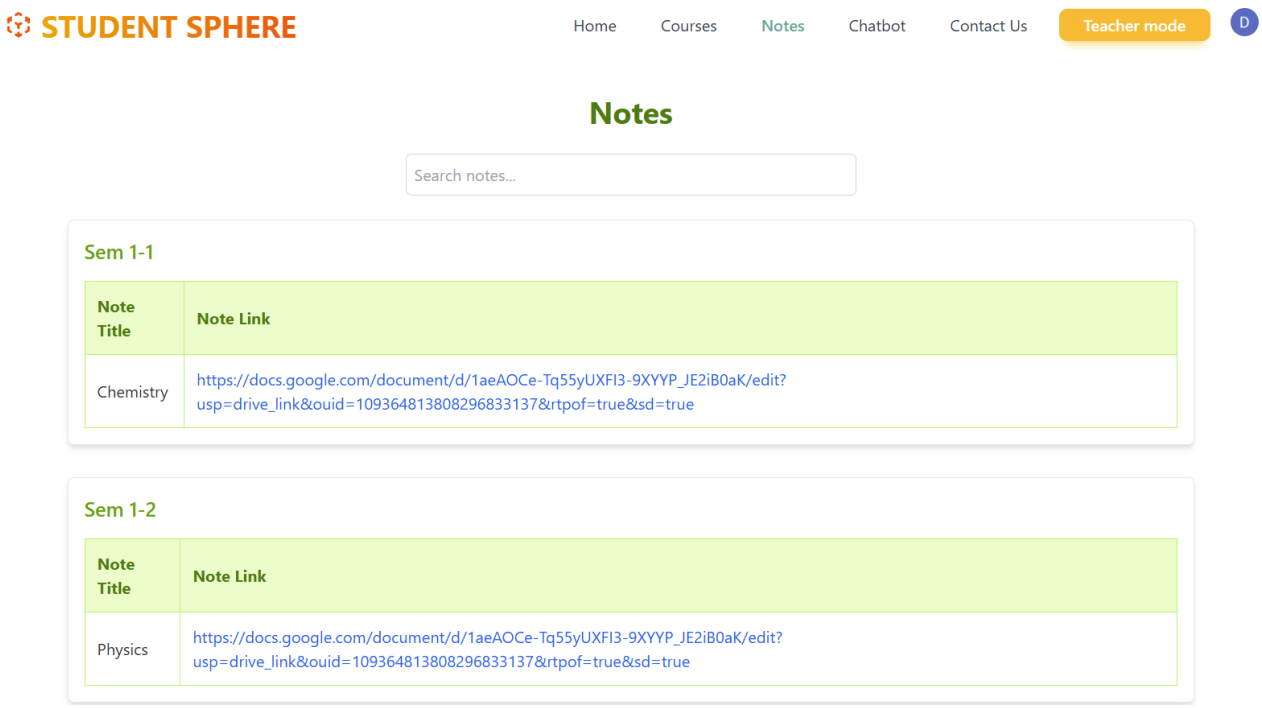
Figno:8.3

Courses Page:



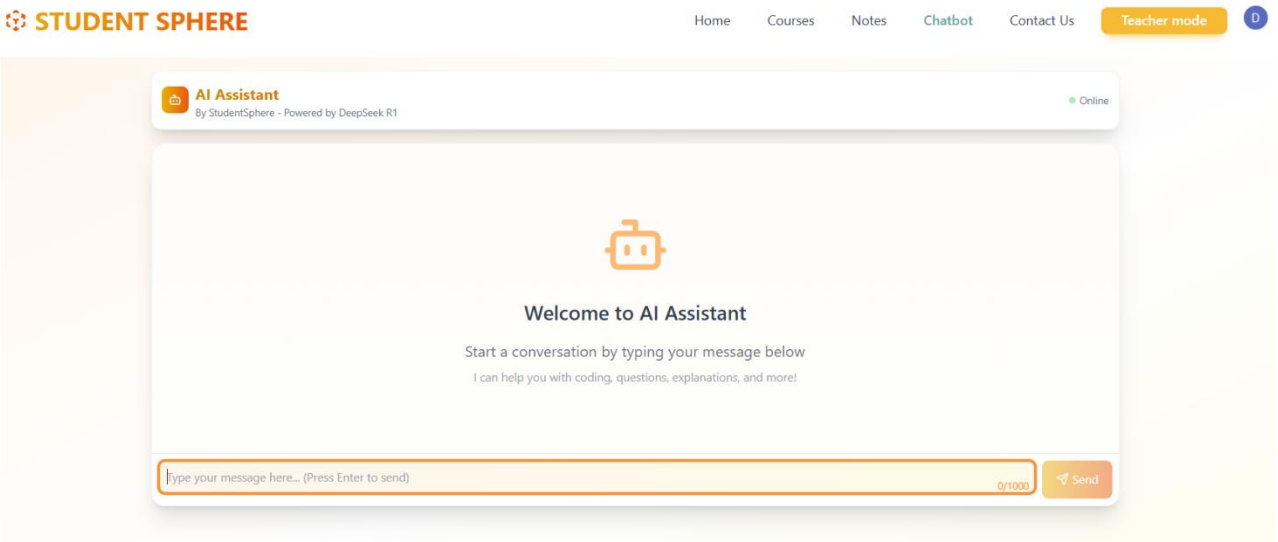
Figno:8.4

Notes Page:



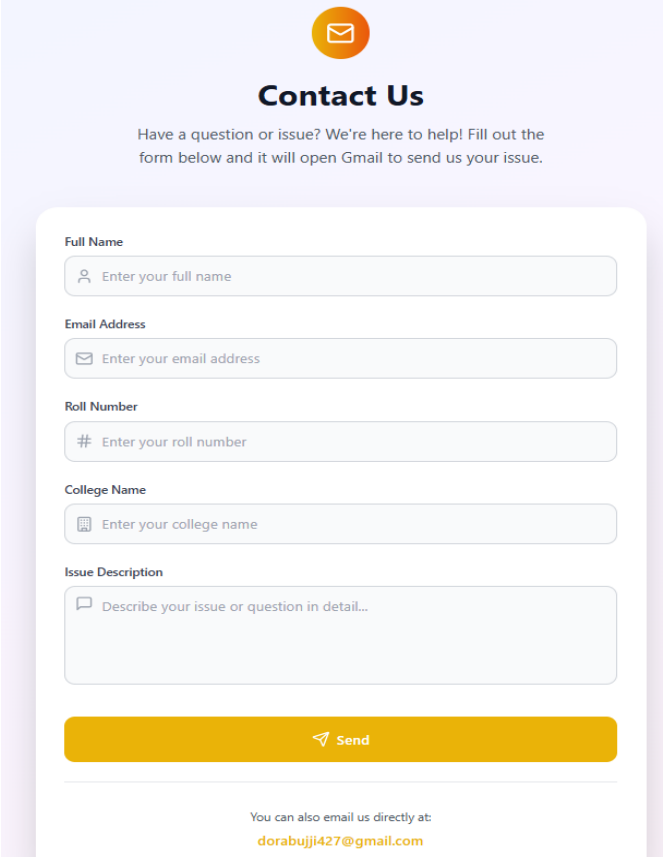
Figno:8.5

Chatbot Page:



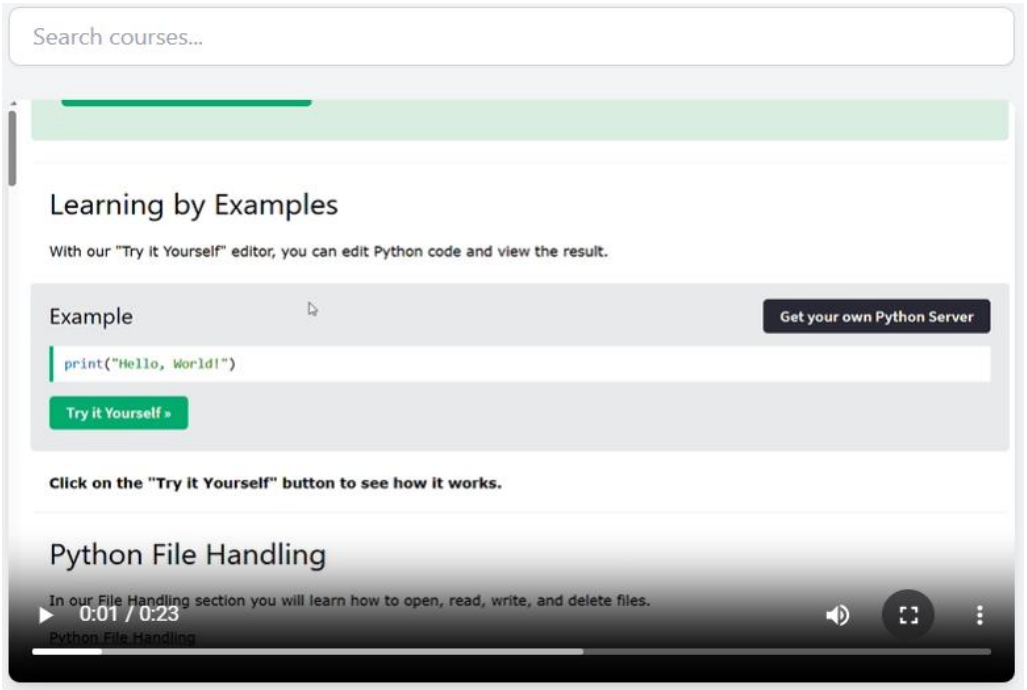
Figno:8.6

Contact Page:



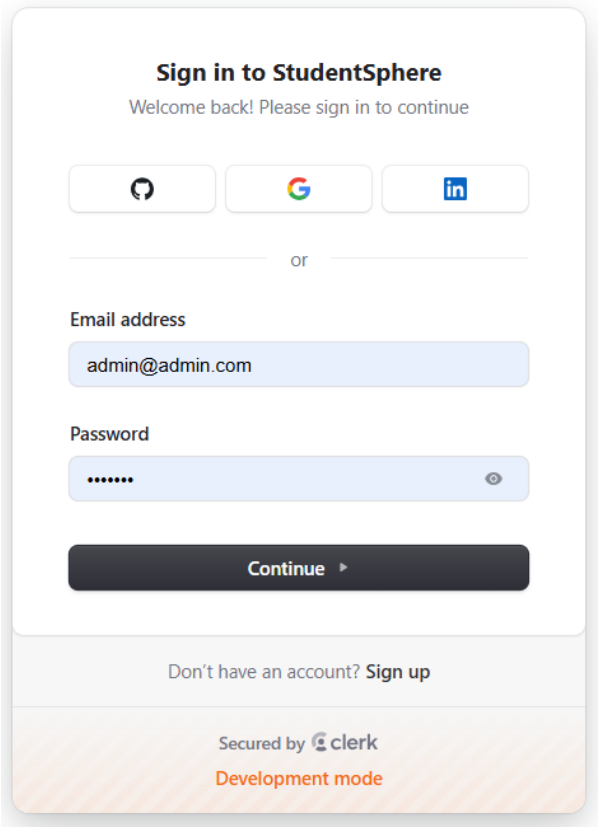
Figno:8.7

Video Page:



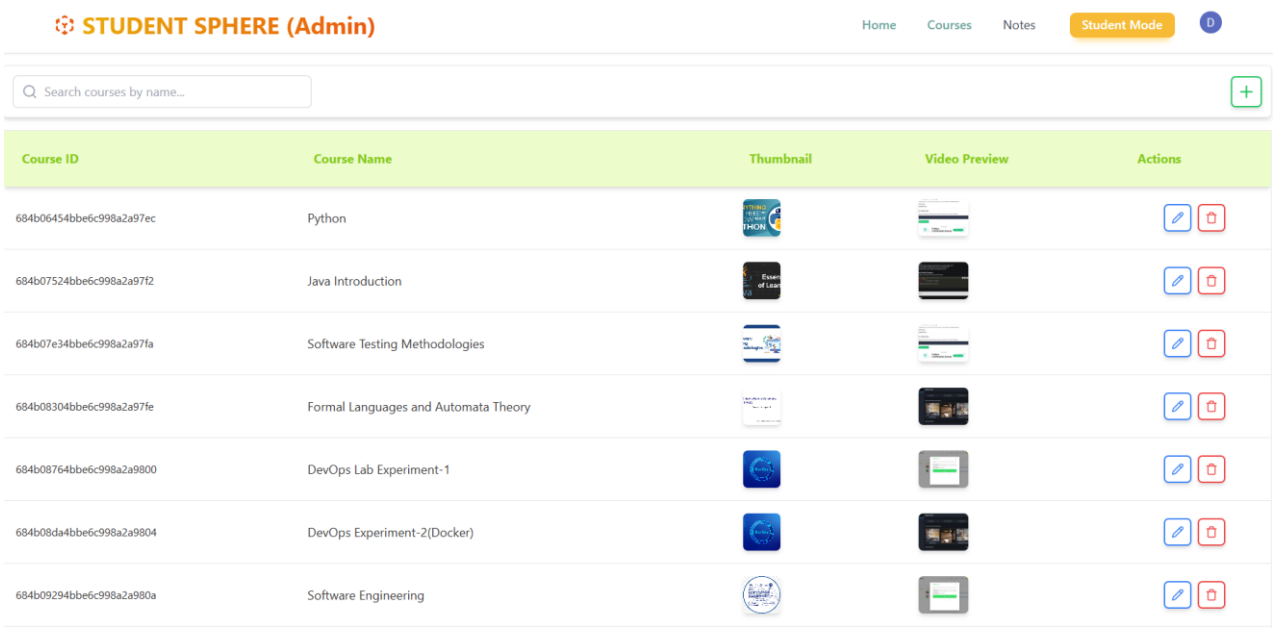
Figno:8.8

Authentication:



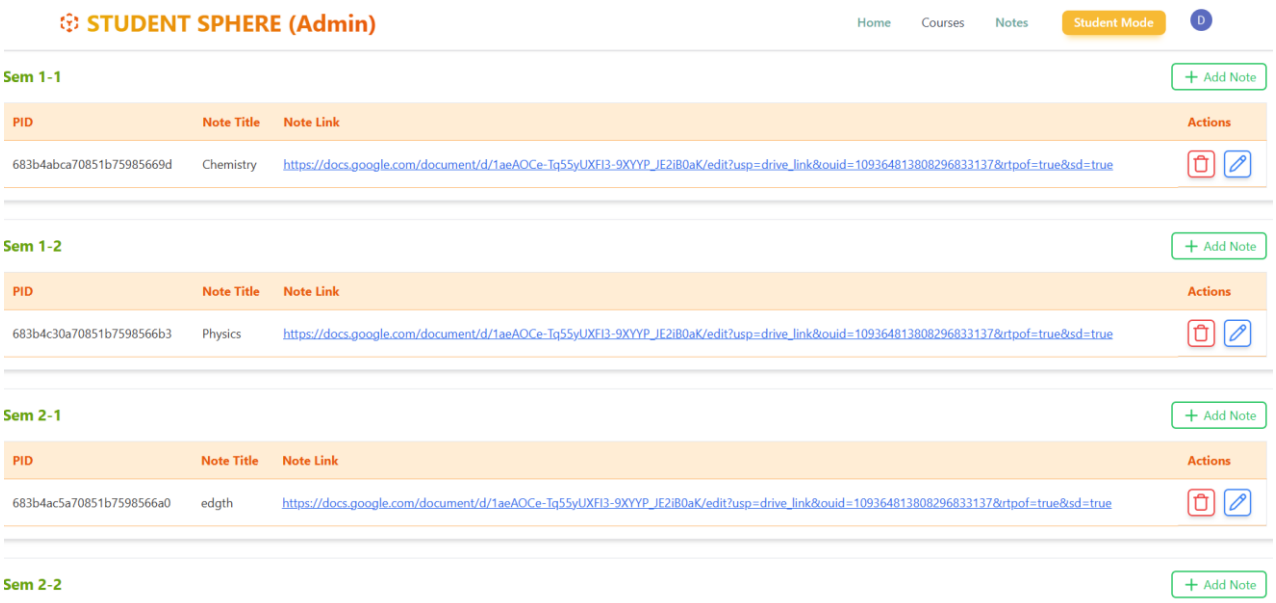
Figno:8.9

Admin Courses Page:



Figno:8.10

Admin Notes Page:



Figno:8.11

Database Clusters:

Figno:8.12

CHAPTER 9
SYSTEM SECURITY

System Security is crucial in ensuring the protection of all computer-based resources including hardware, software, data, procedures, and users from unauthorized access and natural disasters. In the context of **StudentSphere**, a student management system developed using ReactJS, JavaScript, HTML, CSS, and MongoDB, system security focuses on four key areas: **Security, Integrity, Privacy, and Confidentiality**.

1. Security

Security ensures the protection of the entire system from unauthorized access, threats, and attacks. It involves technical implementations and best practices to safeguard data and functionality.

Measures Implemented in StudentSphere:

- **User Authentication** using JWT (JSON Web Token) to ensure secure login.
- **Role-Based Access Control (RBAC)** to restrict features based on user roles (e.g., student, teacher, admin).
- **HTTPS Protocol** enforced for secure data transmission.
- **Firewall & Rate Limiting** to prevent brute-force and denial-of-service attacks.
- **Dependency Auditing** using tools like npm audit to identify and fix vulnerable packages.

2. Integrity

System Integrity ensures that data and system functionalities remain accurate, consistent, and tamper-proof throughout their lifecycle.

Measures Implemented in StudentSphere:

- **Input Validation and Sanitization** to prevent injection attacks and maintain data quality.
- **Password Hashing** with bcrypt for secure storage of user credentials.
- **Atomic Operations** to ensure full completion or rollback of critical operations (e.g., submissions).
- **Mongoose Schema Validation** to enforce data integrity in MongoDB.

3. Privacy

Privacy refers to the user's and organization's control over what information is shared and how it is used.

Measures Implemented in StudentSphere:

- **Minimal Data Collection** to gather only necessary student and user information.
- **User Consent and Notices** during registration, explaining how data is used.
- **Privacy Policy** displayed within the application for user awareness.

CHAPTER 10
CONCLUSION

- StudentSphere offers a complete and user-friendly platform that streamlines academic workflows, improving efficiency in managing learning materials and student interactions.
- Developed using ReactJS, JavaScript, HTML, CSS, and MongoDB, the platform ensures responsiveness, scalability, and fast performance across devices.
- Strong emphasis was placed on system security, data integrity, privacy, and confidentiality, using industry-standard practices to safeguard sensitive educational data from unauthorized access and threats.
- The project followed the Agile methodology, allowing for iterative development with regular feedback, ensuring the system evolved in alignment with actual user needs and expectations.
- Designed with a clean and intuitive UI/UX, StudentSphere allows both students and faculty to easily navigate and interact with its features, reducing learning curves and increasing adoption.
- A key functionality of StudentSphere is centralized access to learning resources:
- Students can seamlessly view and download **video lectures** and **study notes**, while the **AI-powered chatbot** helps them get instant answers to academic queries or platform-related assistance, enhancing the overall learning experience.
- By simplifying access to educational content and fostering responsive support, StudentSphere enhances communication and provides a collaborative and engaging learning environment for students and faculty.

CHAPTER 11

FURTHER ENHANCEMENTS

To ensure **StudentSphere** remains a secure, accessible, and forward-thinking student management system, the following enhancements are proposed. These improvements aim to reinforce the security framework, expand accessibility, enrich assessment methodologies, and incorporate advanced technologies, thereby elevating the platform's overall performance and user engagement.

Proposed Enhancements for StudentSphere

1. Enhanced User Engagement

Personalized Dashboard

- Display recently watched videos, downloaded notes, and chatbot interactions.
- Recommend content based on user behaviour using a basic recommendation system.

Favourites & Bookmarks

- Allow users to bookmark important videos and notes for quick access.

2. Advanced Search and Filtering

Intelligent Search

- Implement fuzzy search and keyword suggestions using libraries like Fuse.js.
- Add filters such as subject, semester, or contributor.

Content Categorization

- Organize videos and notes by subject, difficulty level, or academic relevance.

3. Assessment & Progress Tracking

Learning Analytics

- Track and visualize user progress such as percentage of content viewed or accessed.
- Monitor login frequency and engagement patterns.

Quiz/Assessment Module

- Add functionality for teachers to create quizzes.
- Store responses and scores in MongoDB and provide automated feedback.

4. Chatbot Improvements

Contextual Chatbot Enhancements

- Maintain previous user interactions for better follow-up responses.
- Add subject-specific query modes or menus for easier navigation.

AI Tutor Mode (Optional)

- Integrate external AI APIs (e.g., GPT) to provide in-depth explanations on academic topics.

5. User Experience & Accessibility

Theme Customization

- Provide options for light/dark mode, font size, and contrast adjustments.

Multi-language Support

- Enable UI language selection and integrate real-time translation for content and chatbot.

Voice Support

- Use Web Speech API for text-to-speech and voice commands.

6. Notifications and Announcements

In-App Notifications

- Notify users about new uploads, platform updates, or system maintenance.

Calendar Integration

- Allow academic deadlines and events to be shared and viewed through an integrated calendar.

7. Admin & Faculty Features

Faculty Upload Portal

- Enable verified faculty members to directly upload and manage videos or notes.

Content Management Dashboard

- Provide administrative access to edit/delete resources and review user ratings.

8. Security and Performance

Authentication & Security Enhancements

- Use JSON Web Tokens (JWT) and implement password reset functionality.
- Add role-based access controls (e.g., student, faculty, admin).

Performance Optimization

- Apply lazy loading and pagination for faster performance on content-heavy pages.

9. Technical Enhancements

Progressive Web App (PWA)

- Enable offline access and allow users to install the platform like a native app.

Cloud Integration (Optional)

- Host videos or documents on services like AWS S3 or Google Cloud Storage to improve load times.

Automated Backups

- Schedule backups of the MongoDB database to ensure data recovery in case of failure.

10. Feedback and Support

Feedback Collection System

- Allow students to rate or comment on videos and notes to help identify valuable content.

CHAPTER 12
BIBLIOGRAPHY

Books:

[1] Ethan Brown – *Learning JavaScript: A Hands-On Guide to the Fundamentals of Modern JavaScript*

[2] Alex Banks & Eve Porcello – *Learning React: Functional Web Development with React and Redux*

[3] Kristina Chodorow – *MongoDB: The Definitive Guide*

Websites & Documentation:

[4] ReactJS Official Documentation

[5] MDN Web Docs – JavaScript, HTML, CSS

[6] MongoDB Documentation

[7] W3Schools – HTML, CSS, JS

[8] Tailwind CSS Documentation

[9] ChatGPT API (for chatbot reference)

[10] GeeksforGeeks

[11] FreeCodeCamp

[12] Google Search Engine – for general troubleshooting and additional learning resources.